Lab Sheet 6: MongoDB Basic commands

Use MongoDB to implement the following DB operations

1. Create a database called ‘vehicles’ and write a MongoDB query to select database as “vehicles”.

```
test> use vehicles
switched to db vehicles
```

2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
admin   40.00 KiB
config  60.00 KiB
local   40.00 KiB
```

3. Create a collection called ‘two_wheelers’. (use capping) and Create a collection called

‘four_wheelers’.

```
vehicles> db.createCollection("two_wheelers", {
|     capped: true,
|     size: 50000,
|     max: 100
| })
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
```

4. Add 5 two-wheeler details to the collection named ‘two_wheelers’. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.two_wheelers.insertMany([
| {
| bike_name: "Activa 6G",
| model: "gearless",
| category: "125cc",
| colors_available: ["red","black","blue"],
| manufacturer: "Honda",
| performance: 8,
| timestamp: new Date("2020-01-15"),
| price: 85000
| },
|
| {
| bike_name: "Pulsar 150",
| model: "gear",
| category: "150cc",
| colors_available: ["black","sport red"],
| manufacturer: "Bajaj",
| performance: 9,
| timestamp: new Date("2019-06-10"),
| price: 110000
| },
|
| {
| bike_name: "Splendor Plus",
| model: "gear",
| category: "100cc",
| colors_available: ["black","blue","red"],
| manufacturer: "Hero",
| performance: 7,
| timestamp: new Date("2018-03-20"),
| price: 75000
| },
|
```

```
| {
| bike_name: "Apache RTR 200",
| model: "gear",
| category: "200cc",
| colors_available: ["red","black"],
| manufacturer: "TVS",
| performance: 9,
| timestamp: new Date("2021-09-01"),
| price: 140000
| },
|
| {
| bike_name: "Jupiter",
| model: "gearless",
| category: "125cc",
| colors_available: ["blue","white","black"],
| manufacturer: "TVS",
| performance: 8,
| timestamp: new Date("2020-11-11"),
| price: 90000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b18a063d0006a2147c2907'),
    '1': ObjectId('69b18a063d0006a2147c2908'),
    '2': ObjectId('69b18a063d0006a2147c2909'),
    '3': ObjectId('69b18a063d0006a2147c290a'),
    '4': ObjectId('69b18a063d0006a2147c290b')
  }
}
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

mongosh mongodb://127.0.0.

```
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name: "Swift",
| model: "own",
| category: "car",
| variants: ["vxi","zxi","petrol"],
| manufacturer: "Maruti",
| performance: 8,
| timestamp: new Date("2022-05-10"),
| price: 700000
| },
|
| {
| vehicle_name: "Innova",
| model: "commercial",
| category: "car",
| variants: ["diesel","zxi"],
| manufacturer: "Toyota",
| performance: 9,
| timestamp: new Date("2021-08-01"),
| price: 1800000
| },
|
| {
| vehicle_name: "Ashok Leyland Truck",
| model: "commercial",
| category: "heavy truck",
| variants: ["diesel"],
| manufacturer: "Ashok Leyland",
| performance: 7,
| timestamp: new Date("2019-04-15"),
| price: 2500000
| },
|
```

```
| {
| vehicle_name: "Eicher Pro",
| model: "commercial",
| category: "mini truck",
| variants: ["diesel"],
| manufacturer: "Eicher",
| performance: 6,
| timestamp: new Date("2020-10-05"),
| price: 1500000
| },
|
| {
| vehicle_name: "Volvo Bus",
| model: "commercial",
| category: "bus",
| variants: ["diesel","luxury"],
| manufacturer: "Volvo",
| performance: 9,
| timestamp: new Date("2023-01-01"),
| price: 9000000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b18a123d0006a2147c290c'),
    '1': ObjectId('69b18a123d0006a2147c290d'),
    '2': ObjectId('69b18a123d0006a2147c290e'),
    '3': ObjectId('69b18a123d0006a2147c290f'),
    '4': ObjectId('69b18a123d0006a2147c2910')
  }
}
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b18a063d0006a2147c2907'),
    bike_name: 'Activa 6G',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2020-01-15T00:00:00.000Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b18a063d0006a2147c2908'),
    bike_name: 'Pulsar 150',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'sport red' ],
    manufacturer: 'Bajaj',
    performance: 9,
    timestamp: ISODate('2019-06-10T00:00:00.000Z'),
    price: 110000
  },
  {
    _id: ObjectId('69b18a063d0006a2147c2909'),
    bike_name: 'Splendor Plus',
    model: 'gear',
    category: '100cc',
    colors_available: [ 'black', 'blue', 'red' ],
    manufacturer: 'Hero',
    performance: 7,
    timestamp: ISODate('2018-03-20T00:00:00.000Z'),
    price: 75000
  },
  {
    _id: ObjectId('69b18a063d0006a2147c290a'),
    bike_name: 'Apache RTR 200',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'red', 'black' ],
```

```
    manufacturer: 'TVS',
    performance: 9,
    timestamp: ISODate('2021-09-01T00:00:00.000Z'),
    price: 140000
  },
  {
    _id: ObjectId('69b18a063d0006a2147c290b'),
    bike_name: 'Jupiter',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'blue', 'white', 'black' ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: ISODate('2020-11-11T00:00:00.000Z'),
    price: 90000
  }
]
```

mongosh mongodb://127.0.0.

```
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b18a123d0006a2147c290c'),
    vehicle_name: 'Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2022-05-10T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b18a123d0006a2147c290d'),
    vehicle_name: 'Innova',
    model: 'commercial',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2021-08-01T00:00:00.000Z'),
    price: 1800000
  },
  {
    _id: ObjectId('69b18a123d0006a2147c290e'),
    vehicle_name: 'Ashok Leyland Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2019-04-15T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b18a123d0006a2147c290f'),
    vehicle_name: 'Eicher Pro',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
```

```
    manufacturer: 'Eicher',
    performance: 6,
    timestamp: ISODate('2020-10-05T00:00:00.000Z'),
    price: 1500000
  },
  {
    _id: ObjectId('69b18a123d0006a2147c2910'),
    vehicle_name: 'Volvo Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel', 'luxury' ],
    manufacturer: 'Volvo',
    performance: 9,
    timestamp: ISODate('2023-01-01T00:00:00.000Z'),
    price: 9000000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Activa 6G', price: 85000 },
  { bike_name: 'Pulsar 150', price: 110000 },
  { bike_name: 'Splendor Plus', price: 75000 },
  { bike_name: 'Apache RTR 200', price: 140000 },
  { bike_name: 'Jupiter', price: 90000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Swift', price: 700000 },
  { vehicle_name: 'Innova', price: 1800000 },
  { vehicle_name: 'Ashok Leyland Truck', price: 2500000 },
  { vehicle_name: 'Eicher Pro', price: 1500000 },
  { vehicle_name: 'Volvo Bus', price: 9000000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({manufacturer:"TVS"})
[
  {
    _id: ObjectId('69b18a063d0006a2147c290a'),
    bike_name: 'Apache RTR 200',
    model: 'gear',
    category: '200cc',
    colors_available: [ 'red', 'black' ],
    manufacturer: 'TVS',
    performance: 9,
    timestamp: ISODate('2021-09-01T00:00:00.000Z'),
    price: 140000
  },
  {
    _id: ObjectId('69b18a063d0006a2147c290b'),
    bike_name: 'Jupiter',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'blue', 'white', 'black' ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: ISODate('2020-11-11T00:00:00.000Z'),
    price: 90000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b18a123d0006a2147c290d'),
    vehicle_name: 'Innova',
    model: 'commercial',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2021-08-01T00:00:00.000Z'),
    price: 1800000
  },
  {
    _id: ObjectId('69b18a123d0006a2147c290e'),
    vehicle_name: 'Ashok Leyland Truck',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2019-04-15T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b18a123d0006a2147c290f'),
    vehicle_name: 'Eicher Pro',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 6,
    timestamp: ISODate('2020-10-05T00:00:00.000Z'),
    price: 1500000
  },
```

```
  {
    _id: ObjectId('69b18a123d0006a2147c2910'),
    vehicle_name: 'Volvo Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel', 'luxury' ],
    manufacturer: 'Volvo',
    performance: 9,
    timestamp: ISODate('2023-01-01T00:00:00.000Z'),
    price: 9000000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details

whose rating is more than 5.

```
vehicles> db.two_wheelers.find(
| {performance:{$gt:5}},
| {bike_name:1, category:1, manufacturer:1, _id:0}
| )
[
  { bike_name: 'Activa 6G', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'Pulsar 150', category: '150cc', manufacturer: 'Bajaj' },
  { bike_name: 'Splendor Plus', category: '100cc', manufacturer: 'Hero' },
  { bike_name: 'Apache RTR 200', category: '200cc', manufacturer: 'TVS' },
  { bike_name: 'Jupiter', category: '125cc', manufacturer: 'TVS' }
]
vehicles> db.four_wheelers.find(
| {performance:{$gt:5}},
| {vehicle_name:1, category:1, manufacturer:1, _id:0}
| )
[
  { vehicle_name: 'Swift', category: 'car', manufacturer: 'Maruti' },
  { vehicle_name: 'Innova', category: 'car', manufacturer: 'Toyota' },
  {
    vehicle_name: 'Ashok Leyland Truck',
    category: 'heavy truck',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Eicher Pro',
    category: 'mini truck',
    manufacturer: 'Eicher'
  },
  { vehicle_name: 'Volvo Bus', category: 'bus', manufacturer: 'Volvo' }
]
vehicles>
```

Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called ‘animal’ and write a MongoDB query to select database as ‘animal’.

```
vehicles> use animal
switched to db animal
```

2. Write a MongoDB query to display all the databases.

```
animal> show dbs
admin     40.00 KiB
config    92.00 KiB
local     40.00 KiB
vehicles  80.00 KiB
```

3. Create a collection called ‘wild_animals’.(use capping) and Create a collection called ‘domestic_animals’.

```
animal> db.createCollection("wild_animals", {
|     capped: true,
|     size: 50000,
|     max: 100
| })
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
```

4. Add 5 wild_animal details to the collection named ‘wild_animals’. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
mongosh mongodb://127.0.0.
animal> db.wild_animals.insertMany([
| {
| animal_name: "Lion",
| nature: "harm",
| favorite_foods: ["meat","deer"],
| care_taker_name: "Ravi",
| life_span: 14,
| timestamp: new Date("2018-03-12"),
| expenses: 50000
| },
|
| {
| animal_name: "Tiger",
| nature: "harm",
| favorite_foods: ["meat","rabbits","deer"],
| care_taker_name: "Suresh",
| life_span: 16,
| timestamp: new Date("2019-07-01"),
| expenses: 60000
| },
|
| {
| animal_name: "Elephant",
| nature: "harmless",
| favorite_foods: ["grass","fruits","sugarcane"],
| care_taker_name: "Mahesh",
| life_span: 60,
| timestamp: new Date("2017-05-20"),
| expenses: 70000
| },
|
| {
| animal_name: "Leopard",
| nature: "harm",
| favorite_foods: ["meat","rabbits"],
| care_taker_name: "Ravi",
| life_span: 12,
| timestamp: new Date("2020-09-10"),
| expenses: 45000
| },
|
| {
| animal_name: "Deer",
| nature: "harmless",
| favorite_foods: ["grass","leaves"],
| care_taker_name: "Kiran",
| life_span: 10,
| timestamp: new Date("2021-02-15"),
| expenses: 20000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b190753d0006a2147c2911'),
    '1': ObjectId('69b190753d0006a2147c2912'),
    '2': ObjectId('69b190753d0006a2147c2913'),
    '3': ObjectId('69b190753d0006a2147c2914'),
    '4': ObjectId('69b190753d0006a2147c2915')
  }
}
```

5. Add 5 domestic-animal details to the collection named ‘domestic_animals’. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
| {
| animal_name: "Dog",
| gender: "male",
| favorite_foods: ["meat","biscuits"],
| animal_petname: "Tommy",
| life_span: 13,
| timestamp: new Date("2022-01-01"),
| expenses: 15000
| },
|
| {
| animal_name: "Cat",
| gender: "female",
| favorite_foods: ["fish","milk"],
| animal_petname: "Kitty",
| life_span: 12,
| timestamp: new Date("2021-08-10"),
| expenses: 12000
| },
|
| {
| animal_name: "Cow",
| gender: "female",
| favorite_foods: ["grass","fodder"],
| animal_petname: "Gauri",
| life_span: 20,
| timestamp: new Date("2019-03-15"),
| expenses: 25000
| },
|
| {
| animal_name: "Goat",
| gender: "male",
| favorite_foods: ["grass","leaves"],
| animal_petname: "Ramu",
| life_span: 15,
| timestamp: new Date("2020-06-11"),
| expenses: 10000
| },
|
| {
| animal_name: "Rabbit",
| gender: "female",
| favorite_foods: ["carrots","grass"],
| animal_petname: "Bunny",
| life_span: 8,
| timestamp: new Date("2023-01-20"),
| expenses: 8000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b190b33d0006a2147c2916'),
    '1': ObjectId('69b190b33d0006a2147c2917'),
    '2': ObjectId('69b190b33d0006a2147c2918'),
    '3': ObjectId('69b190b33d0006a2147c2919'),
    '4': ObjectId('69b190b33d0006a2147c291a')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b190753d0006a2147c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2018-03-12T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b190753d0006a2147c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits', 'deer' ],
    care_taker_name: 'Suresh',
    life_span: 16,
    timestamp: ISODate('2019-07-01T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b190753d0006a2147c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits', 'sugarcane' ],
    care_taker_name: 'Mahesh',
    life_span: 60,
    timestamp: ISODate('2017-05-20T00:00:00.000Z'),
    expenses: 70000
  },
  {
    _id: ObjectId('69b190753d0006a2147c2914'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits' ],
    care_taker_name: 'Ravi',
    life_span: 12,
    timestamp: ISODate('2020-09-10T00:00:00.000Z'),
    expenses: 45000
  },
  {
    _id: ObjectId('69b190753d0006a2147c2915'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'leaves' ],
    care_taker_name: 'Kiran',
    life_span: 10,
    timestamp: ISODate('2021-02-15T00:00:00.000Z'),
    expenses: 20000
  }
]
```

```
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69b190b33d0006a2147c2916'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'biscuits' ],
    animal_petname: 'Tommy',
    life_span: 13,
    timestamp: ISODate('2022-01-01T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69b190b33d0006a2147c2917'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 12,
    timestamp: ISODate('2021-08-10T00:00:00.000Z'),
    expenses: 12000
  },
  {
    _id: ObjectId('69b190b33d0006a2147c2918'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass', 'fodder' ],
    animal_petname: 'Gauri',
    life_span: 20,
    timestamp: ISODate('2019-03-15T00:00:00.000Z'),
    expenses: 25000
  },
  {
    _id: ObjectId('69b190b33d0006a2147c2919'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'grass', 'leaves' ],
    animal_petname: 'Ramu',
    life_span: 15,
    timestamp: ISODate('2020-06-11T00:00:00.000Z'),
    expenses: 10000
  },
  {
    _id: ObjectId('69b190b33d0006a2147c291a'),
    animal_name: 'Rabbit',
    gender: 'female',
    favorite_foods: [ 'carrots', 'grass' ],
    animal_petname: 'Bunny',
    life_span: 8,
    timestamp: ISODate('2023-01-20T00:00:00.000Z'),
    expenses: 8000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
animal> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 70000 },
  { animal_name: 'Leopard', expenses: 45000 },
  { animal_name: 'Deer', expenses: 20000 }
]
animal> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Dog', expenses: 15000 },
  { animal_name: 'Cat', expenses: 12000 },
  { animal_name: 'Cow', expenses: 25000 },
  { animal_name: 'Goat', expenses: 10000 },
  { animal_name: 'Rabbit', expenses: 8000 }
]
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find({life_span:12})
[
  {
    _id: ObjectId('69b190b33d0006a2147c2917'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 12,
    timestamp: ISODate('2021-08-10T00:00:00.000Z'),
    expenses: 12000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular

care_taker

```
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69b190753d0006a2147c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2018-03-12T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b190753d0006a2147c2914'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbits' ],
    care_taker_name: 'Ravi',
    life_span: 12,
    timestamp: ISODate('2020-09-10T00:00:00.000Z'),
    expenses: 45000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details

whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbits', 'deer' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'fruits', 'sugarcane' ],
    expenses: 70000
  },
  {
    animal_name: 'Leopard',
    favorite_foods: [ 'meat', 'rabbits' ],
    expenses: 45000
  },
  {
    animal_name: 'Deer',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 20000
  }
]
```

```
animal> db.domestic_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'biscuits' ],
    expenses: 15000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 12000
  },
  {
    animal_name: 'Cow',
    favorite_foods: [ 'grass', 'fodder' ],
    expenses: 25000
  },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 10000
  },
  {
    animal_name: 'Rabbit',
    favorite_foods: [ 'carrots', 'grass' ],
    expenses: 8000
  }
]
animal>
```